

Topic in Module 1 — Cluster and Grid Computing

Course	Modern Computing and Python Programming (26ESCC106)
Module	Module 1 — Modern Computing Approaches
Topic coverage	Cluster and Grid Computing — Introduction and Characteristics
Position in module	After Distributed Computing; before Cloud Computing
Mapped CO	CO1 — Describe modern computing approaches and their characteristics
Bloom's level	L1 — Remember, L2 — Understand
Compiled by	Mr. Manjunath Prasad H. R., Department of AI&ML, MITE
Reviewed by	

1. Introduction

The previous topic established that a distributed system is a collection of independent computers that presents itself to its users as a single coherent system [2]. That definition permits many different arrangements, and the differences between them matter. Two of these arrangements – clusters and grids – are the subject of the topic in the module. They are important in their own way, and they are also the foundation on which cloud computing, the next topic in this module, is built.

Both answer the same question: how do you obtain more computing capacity than one machine can provide? Their answers differ in one respect above all others, and it is worth stating plainly at the outset because every other difference follows from it.

A cluster is many computers owned and administered by one organisation, in one place, connected by a dedicated network. A grid is many computers owned and administered by different organisations, in different places, connected by the public network. Ownership is the distinction; everything else is a consequence of it.

2. Cluster Computing

2.1 Definition

A computer cluster is a set of complete, standalone computers connected by a dedicated high-speed local network and working together as a single integrated computing resource [1, 3]. Each machine in the cluster — called a node — has its own processor, memory, storage and operating system, and could function independently. What makes the collection a cluster is that software coordinates the nodes so that a user submits work to the cluster as a whole rather than to any particular machine.

Figure 1.1 illustrates the basic idea. The input data is given to the root (master) node, which divides the work and distributes the parts to the slave (worker) nodes. The nodes work on their parts in parallel and exchange intermediate results with one another over the network where the problem requires it. The root node then collects the partial results and produces the final output.

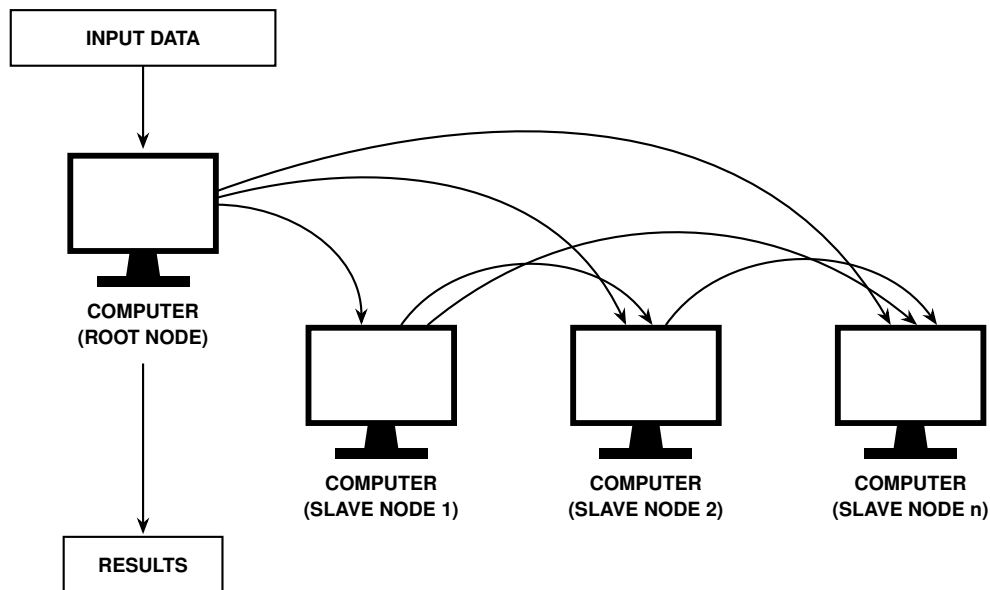


Figure 1.1: *Basic working model of cluster computing*

2.2 Why Clusters Exist

Until the early 1990s, an organisation needing very high computational capacity bought a supercomputer: a custom-designed machine with specially built processors and memory systems, produced in small numbers and priced accordingly. Meanwhile, mass-produced personal computer components were improving rapidly and falling in price, precisely because they were mass-produced.

The cluster is the consequence of noticing that a hundred ordinary machines cost far less than one extraordinary machine of comparable aggregate capacity. The idea was demonstrated in 1994 at NASA's Goddard Space Flight Center, where Thomas Sterling and Donald Becker built a working parallel machine from sixteen commodity personal computers and standard Ethernet running free software. The arrangement became known as a Beowulf cluster [4], and it established that serious parallel computing did not require specialised hardware.

The trade-off is real and should be understood. Commodity components are cheap but their interconnect is slower relative to processor speed than in a purpose-built supercomputer. A cluster therefore performs well on problems that divide into parts needing little communication between them, and less well on problems requiring constant exchange of intermediate results.

2.3 Components of a Cluster

Component	Function
Compute nodes	Standalone computers that perform the actual work; each has its own processor, memory and operating system
Head or master node	The single point of entry through which users log in and submit work; it does not usually perform computation itself
High-speed interconnect	A dedicated private network joining the nodes — typically 10-, 25- or 100-Gigabit Ethernet, or InfiniBand where very low latency is required
Cluster middleware	Software that makes the collection behave as one machine, providing the single system image and handling node failure
Resource manager and job scheduler	Accepts submitted jobs, queues them, allocates nodes and manages execution — for example Slurm or PBS
Parallel programming environment	Libraries that let one program run across many nodes, most commonly MPI for communication between nodes and OpenMP within a node
Shared storage	A file system visible to every node, so that a job can read its input and write its output regardless of where it runs

2.4 Cluster Architecture

Figure 1.2 shows the generic architecture of a cluster, as presented by Yeo, Buyya *et al.* [3]. It is read from the bottom up. The nodes — PCs or workstations — are joined by the cluster interconnection network or switch. Each node has network interface hardware that attaches it to this network, communication software that provides fast and reliable data exchange between nodes, and its own operating system. Above the individual nodes sits the cluster middleware, which provides the single system image and the availability infrastructure: it makes the separate machines appear as one computer and keeps the service running when a node fails. On top of the middleware, sequential applications run directly, while parallel applications run through a parallel programming environment — compilers, message-passing libraries such as MPI, debuggers and performance tools — that lets one program use many nodes at once.

Figure 1.3 shows how such a cluster is typically deployed in practice. Users never connect to the compute nodes directly; they log in to the head node over the campus network or the Internet and submit their jobs to the scheduler running there. The head node, the shared storage and every compute node are joined by the private high-speed interconnect, which carries the messages exchanged between the parts of a running job as well as the traffic between each node and the shared file system. The head node and the shared storage are where much of the middleware and scheduling software of Figure 1.2 actually runs. Everything inside the dashed boundary belongs to one organisation and sits in one facility.

2.5 Single System Image

The property that distinguishes a cluster from a merely networked room of computers is the single system image: the cluster presents itself to the user as one machine. The user does not choose which nodes will

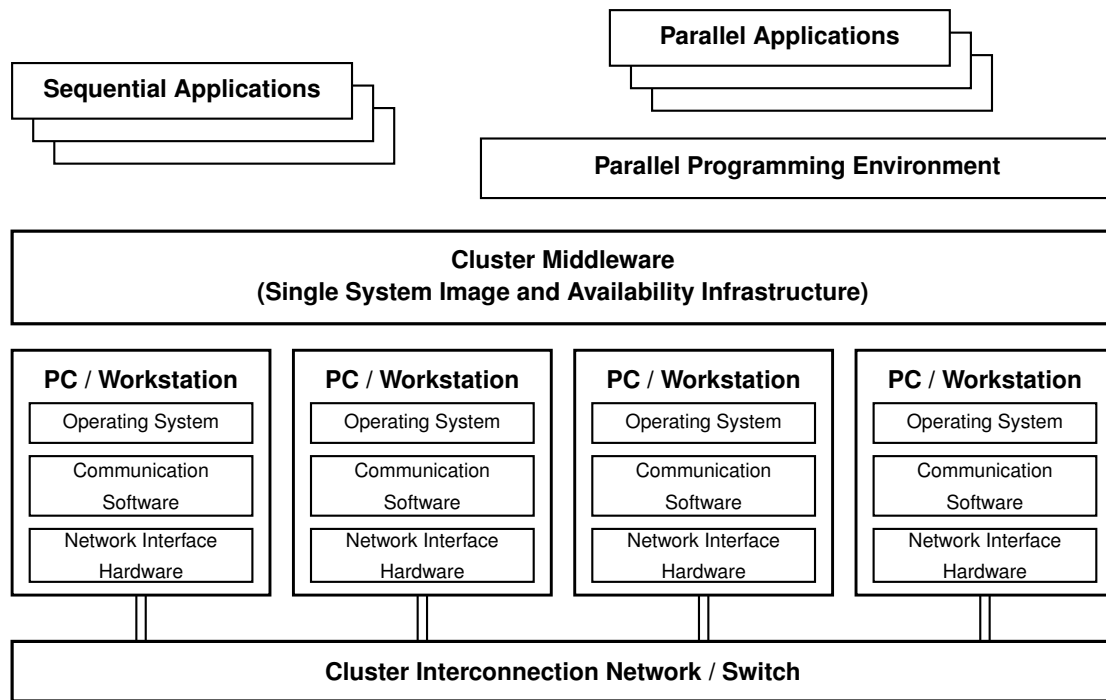


Figure 1.2: Cluster computer architecture (redrawn from [3]; the figure originates in [7])

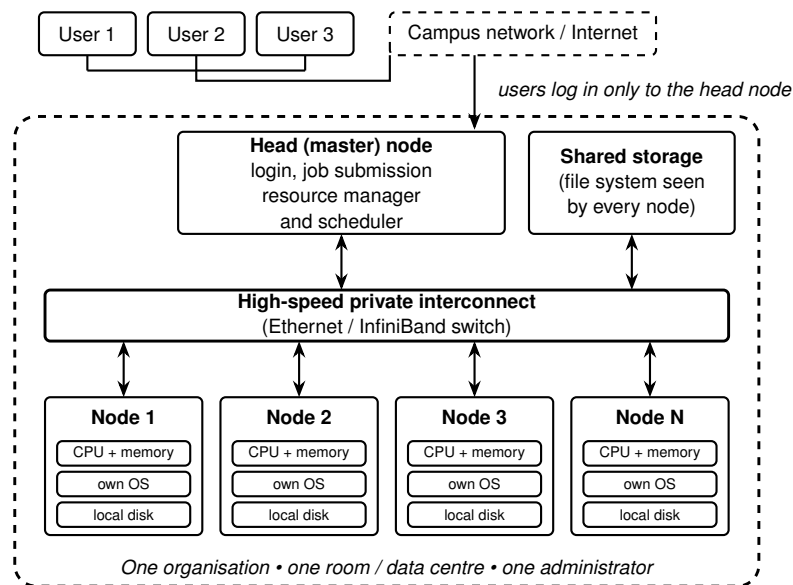


Figure 1.3: Physical organisation of a cluster

run their program, does not log in to individual machines, and does not manually distribute data. They describe the resources their job needs and submit it; the scheduler decides where it runs. This abstraction is what makes a cluster usable, and providing it is the main job of the cluster middleware [3, 1].

2.6 Characteristics of Cluster Computing

The following are the characteristics of cluster computing:

- **Single administrative domain** — all nodes are owned, funded and administered by one organisation, under one security policy

- **Physical co-location** — nodes sit in one room or one data centre, typically in the same rack or set of racks
- **Homogeneity** — nodes are usually identical or near-identical in hardware, operating system and configuration, which simplifies scheduling and programming
- **Tight coupling** — a dedicated, private, high-bandwidth and low-latency interconnect, not shared with general network traffic
- **Dedicated resources** — nodes exist to serve the cluster and are not used for anything else
- **Centralised scheduling** — one resource manager has complete knowledge of and authority over every node
- **Single system image** — the collection appears to the user as one computer
- **High availability** — the failure of one node need not stop the cluster, because work can be restarted elsewhere
- **Incremental scalability** — capacity grows by adding nodes, within the limits of space, power and interconnect

2.7 Types of Clusters

Clusters are built for three broad purposes [1].

Type	Purpose	Typical use
High-performance (HPC) cluster	Maximise computational throughput on a single large problem by dividing it across nodes	Scientific simulation, weather modelling, molecular dynamics, model training
Load-balancing cluster	Distribute many independent incoming requests across nodes so that no node is overwhelmed	Web servers, application servers, online services
High-availability (failover) cluster	Maintain service continuity by having standby nodes take over when an active node fails	Database servers, banking and other systems that must not stop

2.8 Examples of Clusters

- Beowulf clusters — the original 1994 NASA machine and the many university and departmental clusters modelled on it, built from commodity PCs running Linux
- The PARAM series developed by C-DAC under the National Supercomputing Mission, for example PARAM Siddhi-AI at C-DAC Pune and PARAM Pravega at IISc Bengaluru — high-performance clusters of many compute nodes joined by a high-speed interconnect [11]
- Most of the world's fastest computers listed in the TOP500 ranking, which are built as clusters rather than as single custom machines [10]

- Web server farms behind a load balancer — search engines, e-commerce and railway or airline booking sites spread incoming requests across many identical servers (load-balancing clusters)
- High-availability database clusters, such as Oracle Real Application Clusters, used for core banking and reservation systems where a standby node must take over if an active node fails

2.9 Advantages and Limitations

Advantages

- Substantially better price-to-performance ratio than a purpose-built supercomputer
- Capacity can be increased in small increments rather than by replacing the whole machine
- Redundancy is inherent, so availability is high if the software is configured for it
- Built from standard components with standard software, so expertise and spare parts are widely available

Limitations

- Bounded by one organisation's budget, floor space, power supply and cooling capacity
- Programs must be written explicitly for parallel execution; existing sequential software does not benefit
- The interconnect eventually limits how far a tightly coupled problem can be scaled
- Capacity is fixed once purchased — it must be sized for peak demand and is idle at other times

3. Grid Computing

3.1 Definition

Grid computing is the coordinated sharing of computing resources that belong to different organisations, in different locations, in order to solve a common problem. The resources are federated rather than owned in common: each participating institution keeps ownership of and authority over its own machines, and contributes their use under agreed terms.

The term was introduced in the late 1990s, principally through the work of Ian Foster and Carl Kesselman [5], who characterised the field by the problem of coordinated resource sharing across institutional boundaries [6].

3.2 The Power Grid Analogy

The name is borrowed from the electrical power grid, and the analogy explains the ambition [5]. When you switch on a light, you do not know or care which generating station supplied the electricity, whether it came from one source or several, or where those sources are. You draw power from a socket, it is metered, and you are billed. The generation, transmission and coordination are invisible.

Grid computing proposed the same relationship with computation: a researcher would draw processing power from a socket without knowing which institution's machines were serving them. It is worth noting

that this ambition was only partially realised. Electricity is a uniform commodity and computing is not — machines differ in architecture, software and policy in ways that generators do not — and the seamlessness the analogy promised proved difficult to deliver. The vision was largely fulfilled later, by cloud computing, and by a different route.

3.3 The Virtual Organisation

The central concept in grid computing is the virtual organisation: a set of individuals or institutions, drawn from different real organisations, who agree to share specified resources under specified rules for a specified purpose. A virtual organisation is defined by its sharing agreement, not by a legal entity or a physical site [6].

A concrete example makes this clear. Physicists at twenty universities in twelve countries collaborate on one experiment. Each university owns a cluster. No university is willing to hand over control of its cluster to any other, and each has its own security policy, its own users and its own local commitments. Yet the experiment needs more capacity than any one of them has. The virtual organisation is the arrangement under which those twenty clusters serve the experiment while each remains under its owner's control. Grid middleware exists to make that arrangement work in practice.

3.4 Grid Architecture

Figure 1.4 shows how the sites of a grid are arranged and how a user's job reaches them. Each site — a university cluster, a research laboratory's supercomputer, a data archive, a scientific instrument, or a pool of contributed desktop PCs — is owned by a different institution. Every site keeps its own local scheduler and its own policy, which decide what may run there, for whom and at what priority; nothing in the grid overrides them. A layer of grid middleware at each site acts as a gateway, translating a request arriving from the grid into a local action that the owner has already agreed to permit. The sites are joined only by the wide-area network, drawn in the middle of the figure, and the dashed boundary around them is the virtual organisation: an agreement, not a building or a company.

The upper part of the figure shows what happens when work is submitted. The user does not approach the sites one by one. They authenticate once with a digital certificate and hand the job to a grid portal or resource broker, which is the only component they deal with directly. The broker asks the information service which resources are currently available and suitable — what exists, what is free, and what matches the job's requirements — because membership of a grid changes without notice. The security service then establishes that this user, authenticated at their home institution, is to be trusted at each site the broker has chosen, so that one login is accepted everywhere (single sign-on). Only then does the broker dispatch the job, and the data it needs, across the network. At the receiving site the middleware hands the job to the local scheduler, which queues it like any local work, and the results travel back the same way. Everything the user would otherwise have to do — find resources, negotiate access at each institution, move data, track progress — is what the middleware and these services exist to do.

Internally, this software is organised in layers [6]. The Fabric layer is the physical resources themselves, each under its local manager. The Connectivity layer provides communication and security between sites. The Resource layer negotiates access to, and controls, one resource at a time, such as starting and

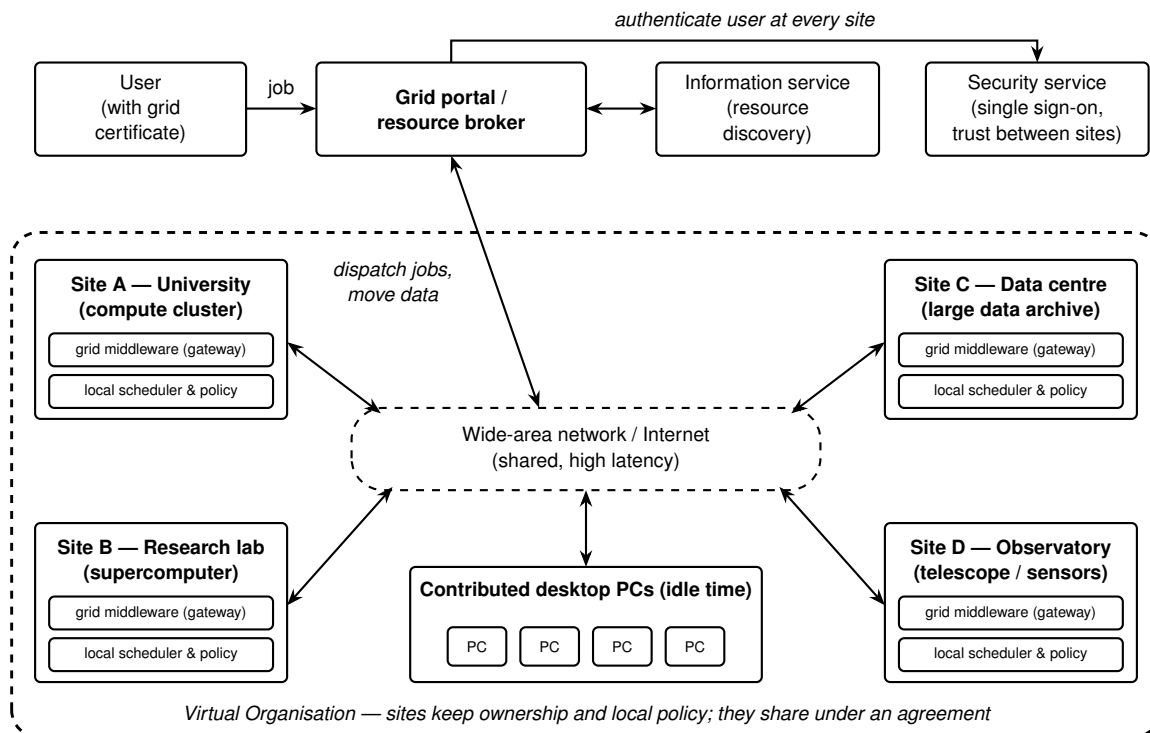


Figure 1.4: Architecture of a grid — independent sites joined into a virtual organisation through grid middleware

monitoring a job on one cluster. The Collective layer coordinates many resources together — discovery, brokering, scheduling across sites and monitoring — and it is where the information service and the resource broker of Figure 1.4 belong. Applications sit on top and use these services. The Globus Toolkit provided software for the Connectivity, Resource and Collective layers.

3.5 Characteristics of Grid Computing

- **Multiple administrative domains** — no single owner or authority; each site retains autonomy over its own resources and enforces its own policies
- **Geographical distribution** — resources are spread across cities, countries or continents
- **Heterogeneity** — nodes differ in hardware architecture, operating system, software stack and configuration
- **Loose coupling** — sites are connected by wide-area networks or the public Internet, with latency orders of magnitude higher than a cluster interconnect
- **Dynamic and unpredictable membership** — resources join, leave and fail without notice, so the grid must tolerate their disappearance
- **Non-dedicated resources** — contributed machines usually have local work of their own, and grid jobs run on spare capacity at lower priority
- **Policy-based sharing** — what may be used, by whom, and for what is governed by negotiated agreements rather than by ownership
- **Decentralised scheduling** — no single scheduler has authority over all resources; allocation is

negotiated between sites

- **Reliance on standard protocols and middleware** — the Globus Toolkit and the Open Grid Services Architecture were the principal efforts to standardise this
- **Best-effort service** — there is generally no guarantee of when a job will run or on what, because no participant has undertaken to provide capacity on demand

3.6 Types of Grids

Grids are classified by what they share [1, 5].

Type	Description
Computational grid	Aggregates processing capacity across sites to execute computationally intensive work
Data grid	Provides controlled, coordinated access to very large datasets held at multiple institutions
Collaboration or service grid	Shares instruments, services and applications — telescopes, sensors, specialised software — rather than raw computation
Desktop or volunteer grid	Uses idle time on ordinary personal computers contributed by the public, coordinated over the Internet

3.7 Examples

- The Worldwide LHC Computing Grid, which distributes and processes data from the Large Hadron Collider across roughly 170 computing centres in more than 40 countries [8]
- The European Grid Infrastructure, federating computing resources across European research institutions [9]
- GARUDA, the Indian national grid initiative coordinated by C-DAC, connecting research and academic institutions across the country [11]
- Volunteer computing projects for protein folding and radio-signal analysis, run on donated capacity from personal computers

3.8 Advantages and Limitations

Advantages

- Exploits existing resources that would otherwise sit idle, at little additional hardware cost
- Aggregate capacity far exceeding what any single participant could afford to build
- Enables collaboration across institutional and national boundaries without transferring ownership
- No participant is required to purchase the whole of the capacity it occasionally needs

Limitations

- Heterogeneity makes software difficult to write, deploy and validate across sites
- Security and trust are hard: a user authenticated at one institution must be granted appropriate rights at another that does not directly know them
- Wide-area latency makes the grid unsuitable for tightly coupled problems requiring frequent communication
- Middleware is complex and demands specialist administration at every participating site
- No guarantee of availability or performance, because resources are contributed rather than committed
- Accounting is difficult — measuring and settling who consumed whose capacity never worked cleanly, which limited participation beyond publicly funded research

4. Cluster Computing versus Grid Computing

The table below draws together the differences established in the two previous sections [1, 6].

Aspect	Cluster computing	Grid computing
Ownership	One organisation	Many independent organisations
Administration	Single administrative domain	Multiple autonomous domains
Location	Co-located in one facility	Geographically distributed
Hardware and software	Homogeneous	Heterogeneous
Interconnect	Dedicated high-speed LAN, low latency	Wide-area network or Internet, high latency
Coupling	Tight	Loose
Resource dedication	Dedicated to the cluster	Shared with the owner's own local workload
Scheduling	Centralised, with full authority	Decentralised and negotiated
Membership	Static and known	Dynamic and variable
Security	One policy, one trust domain	Cross-domain trust and identity federation required
Service guarantee	Predictable and controlled	Best effort
User view	A single machine	A pool of resources reached through middleware
Suitable workload	Tightly coupled parallel problems	Independent, coarse-grained tasks

5. Review Questions

CO1 — Describe modern computing approaches and their characteristics

Bloom's Level L1 — Remember, L2 — Understand

1. Define cluster computing and explain what distinguishes a cluster from a set of networked computers in the same room.
2. List the principal components of a cluster and state the function of each.
3. Explain the term single system image and describe why it matters to a cluster user.
4. List any six characteristics of cluster computing.
5. With a neat diagram, explain the architecture of a computer cluster, showing both its physical organisation and its software layers.
6. With a neat diagram, explain how a user's job is executed on a grid, stating the role of the resource broker, the information service, the security service and the grid middleware at each site.
7. Differentiate between high-performance, load-balancing and high-availability clusters, giving one application of each.
8. State any four limitations of cluster computing.
9. Explain the power grid analogy. Identify one respect in which the analogy does not hold.
10. What is a virtual organisation? Explain with an example why it is the central concept in grid computing.
11. List any six characteristics of grid computing.
12. Differentiate between a computational grid and a data grid.
13. Compare cluster and grid computing with respect to ownership, coupling, homogeneity, scheduling and service guarantee.
14. Explain why security is substantially harder to provide in a grid than in a cluster.
15. Why is grid computing unsuitable for tightly coupled parallel problems?
16. Give one real-world example each of a high-performance, a load-balancing and a high-availability cluster.

6. References and Further Reading

Items 1 to 7 are the sources used in preparing these notes and are suggested for further reading; section and chapter numbers are indicative. Items 8 to 11 are the project and ranking sites from which the examples and figures quoted in the text were taken.

- [1] K. Hwang, G. C. Fox and J. J. Dongarra, *Distributed and Cloud Computing: From Parallel Processing to the Internet of Things*, Morgan Kaufmann, 2012 — Chapter 2 on computer clusters for scalable parallel computing, and Chapter 7 on grid computing systems and resource management.

- [2] R. Buyya, C. Vecchiola, S. T. Selvi, S. Poojara and S. N. Srirama, *Mastering Cloud Computing*, 2nd Edition, Morgan Kaufmann, 2026 — Chapter 2 on the principles of parallel and distributed computing.
- [3] C. S. Yeo, R. Buyya, H. Pourreza, R. Eskicioglu, P. Graham and F. Sommers, “Cluster Computing: High-Performance, High-Availability, and High-Throughput Processing on a Network of Computers”, in A. Y. Zomaya (ed.), *Handbook of Nature-Inspired and Innovative Computing*, Springer, 2006 — the source of Figure 1.2.
- [4] T. Sterling, D. J. Becker, D. Savarese, J. E. Dorband, U. A. Ranawake and C. V. Packer, “BE-OWULF: A Parallel Workstation for Scientific Computation”, *Proceedings of the 24th International Conference on Parallel Processing*, 1995.
- [5] I. Foster and C. Kesselman (eds.), *The Grid: Blueprint for a New Computing Infrastructure*, 2nd Edition, Morgan Kaufmann, 2004.
- [6] I. Foster, C. Kesselman and S. Tuecke, “The Anatomy of the Grid: Enabling Scalable Virtual Organizations”, *International Journal of High Performance Computing Applications*, vol. 15, no. 3, pp. 200–222, 2001 — the source of the virtual organisation and of the layered grid architecture.
- [7] R. Buyya (ed.), *High Performance Cluster Computing: Architectures and Systems*, Volume 1, Prentice Hall PTR, 1999 — where the architecture of Figure 1.2 first appeared.
- [8] Worldwide LHC Computing Grid, CERN. <https://wlcg.web.cern.ch/>
- [9] EGI Federation (European Grid Infrastructure). <https://www.egi.eu/>
- [10] TOP500 list of the world’s most powerful computer systems. <https://www.top500.org/>
- [11] Centre for Development of Advanced Computing (C-DAC) — PARAM systems under the National Supercomputing Mission, and the GARUDA national grid initiative. <https://www.cdac.in/>

Topic in Module 1 — Edge Computing

Course	Modern Computing and Python Programming (26ESCC106)
Module	Module 1 — Modern Computing Approaches
Topic coverage	Edge Computing — Introduction, Need, Architecture, Cloud vs. Edge, Applications
Position in module	After Cloud Computing; last topic of the module
Mapped CO	CO1 — Describe modern computing approaches and their characteristics
Bloom's level	L1 — Remember, L2 — Understand
Compiled by	Mr. Manjunath Prasad H. R., Department of AI&ML, MITE
Reviewed by	

1. Introduction

1.1 Definition

Edge computing means performing computation close to where the data is produced, instead of sending all of it to a distant data centre [4, 1]. The “edge” is the outer boundary of the network: the place where cameras, sensors, machines, vehicles and phones meet the network, as opposed to the centre, where the large cloud data centres sit.

Two terms are worth fixing before going further, because the whole topic turns on them.

- **Latency** is the time taken for data to travel from where it is produced to where it is processed, and for the answer to come back. It is measured in milliseconds (ms), and one millisecond is a thousandth of a second.
- **Bandwidth** is how much data a network link can carry per second, measured in megabits per second (Mbps). A link with higher bandwidth carries more data in the same time.

In the cloud model studied in the previous topic, data travels from the device to a data centre that may be hundreds or thousands of kilometres away, is processed there, and the result travels back. Edge computing places a computer — a gateway, a small server, or the device itself — beside the data source, so that most of the work finishes locally and only a small part of the data ever travels to the cloud. Figure 2.1 shows this basic arrangement.

Edge computing does not replace cloud computing. The two work together: the edge handles what must be fast, private or independent of the network, and the cloud handles what needs large storage, heavy computation or a view across many sites.

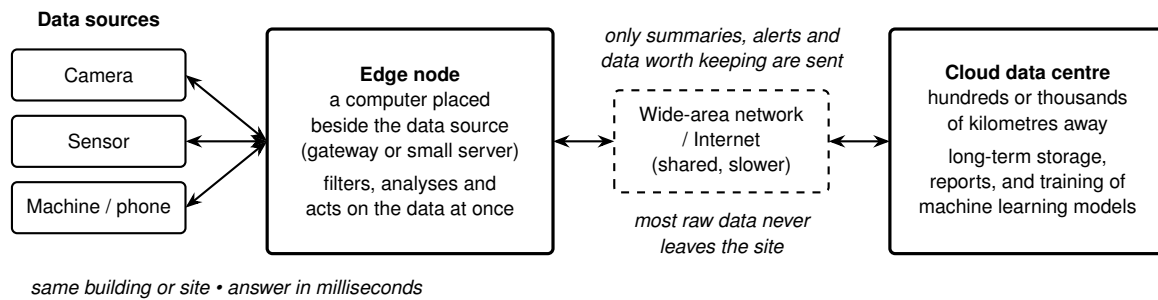


Figure 2.1: Basic working model of edge computing

1.2 Where the Edge Is

“The edge” is not one fixed place. Depending on the application the computing may sit at any of the following points, and a real system often uses more than one of them [1].

- **On the device itself** — a camera that recognises a number plate inside the camera, or a watch that counts steps without help from a phone
- **On a gateway at the site** — one small computer in a factory, farm or hospital that serves all the devices around it
- **At the network operator’s premises** — a small data centre at a mobile base station, serving every user connected to it. This arrangement is called multi-access edge computing [7]

The term **fog computing** is often met alongside edge computing. It describes the same idea with the emphasis on an intermediate layer of gateways and regional nodes between the devices and the cloud, rather than on the devices themselves [6]. For this course the two may be treated as the same approach seen from slightly different distances.

2. Need for Edge Computing

Cloud computing solved the problem of obtaining capacity without owning it. It did not solve the problem of distance. Sending data to a distant data centre takes time, consumes network capacity, and fails when the network fails. The reasons for computing at the edge follow from these facts [4, 5].

- **Latency** — some decisions cannot wait for a round trip to a data centre. A self-driving vehicle that must brake, a robot arm that must stop before it injures someone, or a game that must respond to a movement, needs an answer within a few milliseconds. A round trip to a cloud region typically costs tens of milliseconds, and more when the network is busy.
- **Bandwidth** — sending everything is often simply too much data. One high-definition camera produces roughly 4 Mbps of video, so a campus with 100 cameras would need about 400 Mbps of upload capacity, continuously, only to send video that is almost entirely uneventful. An edge node can watch all 100 streams locally and send on only the few seconds that matter.
- **Intermittent connectivity** — ships, mines, farms, oil rigs and rural clinics do not have a dependable link to the Internet. Work that depends on reaching the cloud stops when the link does; work done at

the edge continues and synchronises later.

- **Privacy and regulation** — medical records, classroom footage and factory process data may be sensitive, or may be required by law to stay within the premises or the country. Processing locally and sending only anonymous summaries keeps the raw data where it belongs.
- **Cost** — network transfer and cloud storage are charged by volume, so discarding uninteresting data at the edge reduces both bills.
- **Scale** — the number of connected devices grows faster than the capacity of the links to the cloud, so a design that forwards everything to the centre becomes harder to sustain as devices are added [1, 3].

3. Edge Computing Architecture

An edge computing system is usually described as a set of tiers, or layers, arranged by distance from the data [1, 6]. Figure 2.2 shows the four tiers and what each one does. It is read from the bottom, where the data is created, to the top, where the cloud sits.

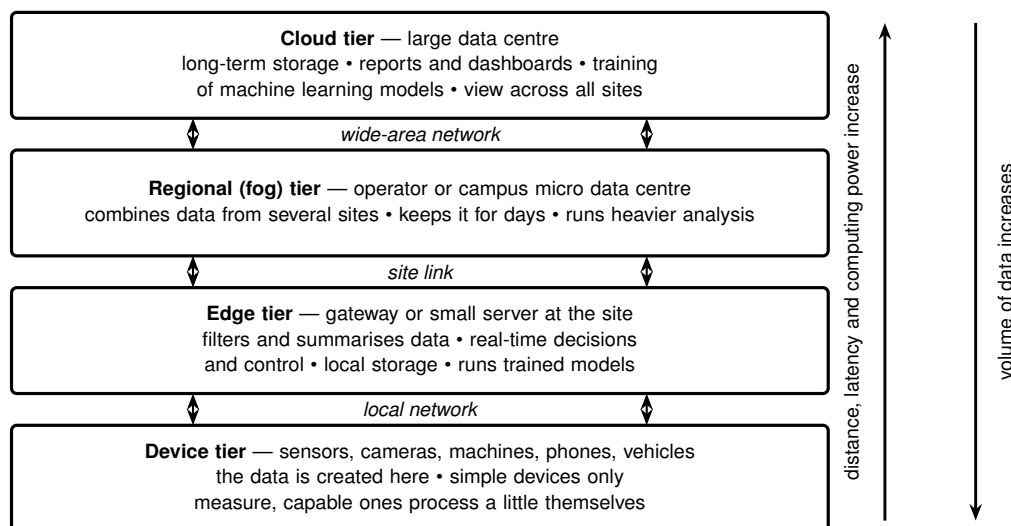


Figure 2.2: Architecture of an edge computing system: four tiers, from the devices that create the data to the cloud

3.1 The Tiers

- **Device tier** — the sensors, cameras, machines, vehicles and phones that produce the data. A simple sensor only measures and reports; a more capable device performs a little processing itself, such as discarding a reading that has not changed.
- **Edge tier** — one or more computers at the site: a gateway, an industrial PC or a small server. This tier receives data from the devices over a local network, filters and summarises it, takes the decisions that must be immediate, keeps a short history locally, and forwards what is worth forwarding.
- **Regional or fog tier** — a micro data centre belonging to the organisation or to the network operator and serving several sites. It combines their data, keeps it for days or weeks, and runs analysis that is too heavy for a gateway. Small systems often leave this tier out and connect the edge tier directly to the cloud.

- **Cloud tier** — the large data centre studied in the previous topic. It provides effectively unlimited storage and computing power, a view across every site, and the place where machine learning models are trained before being sent down to the edge to be used.

3.2 How Work Is Divided Between the Tiers

Two quantities change as one moves up the diagram, and the design follows from them. The volume of data is largest at the bottom, because every device produces data continuously, and it shrinks at each tier as data is filtered and summarised. Computing power, storage and distance all grow towards the top. The working rule is therefore simple: perform a task at the lowest tier that is able to perform it, and send upward only what the higher tier genuinely needs [4].

A traffic camera illustrates the division. The camera captures video (device tier). The edge node watches that video, recognises a vehicle crossing a red light and raises an alert within a fraction of a second, discarding the rest (edge tier). Each evening it uploads the recorded violations and a count of vehicles per hour to the cloud, which stores them, prepares reports for the whole city, and periodically trains an improved recognition model that is then sent back to the edge nodes (cloud tier).

4. Cloud versus Edge Computing

The two are partners rather than rivals, but they differ in almost every practical respect [4, 2].

Aspect	Cloud computing	Edge computing
Location of processing	Large data centres, far from the user	Close to where the data is produced
Typical response time	Tens of milliseconds or more	A few milliseconds
Computing power	Very large and elastic	Limited by what fits at the site
Storage	Effectively unlimited	Small; usually a short local history
Data sent over the network	All of it	Only summaries and important events
Works without the Internet	No	Yes, for local tasks
Privacy of raw data	Leaves the premises	Can stay on the premises
Cost pattern	Pay for capacity, transfer and storage used	Pay once for the equipment, then little transfer cost
Scope of view	Every site together	One site or one device
Suitable work	Training models, long-term storage, reports across sites	Immediate decisions, filtering, local control
Management	One provider manages the hardware	Many small units in many places must be managed

5. Applications

- **Industrial automation** — machines on a production line are monitored for vibration, temperature and current, and an edge node stops a machine or raises a maintenance alert before a fault becomes a breakdown
- **Autonomous and assisted driving** — a vehicle interprets its cameras and sensors on board, because braking cannot wait for a reply from a data centre [5]
- **Video surveillance and traffic management** — cameras and edge nodes detect events locally and send only the relevant clips instead of streaming everything
- **Healthcare** — bedside monitors raise alarms locally, and patient data is summarised before anything leaves the hospital
- **Agriculture** — soil, weather and crop-health sensors are processed on a farm gateway, which keeps working where network coverage is poor
- **Smart cities and utilities** — street lighting, water distribution and electricity meters are controlled locally, with readings summarised for the central system
- **Retail** — shelf cameras, billing counters and stock sensors are processed in the store, so that it continues to operate even if the link to the head office is lost

- **Content delivery and gaming** — copies of popular videos, pages and game servers are kept near users so that they load quickly and the central servers carry less load

6. Advantages and Limitations

Advantages

- Responses in milliseconds, because the data does not travel far
- Much less network traffic, and lower transfer and storage cost
- Continues to work when the connection to the cloud is lost
- Sensitive data can be kept within the premises
- Capacity grows naturally as devices and sites are added

Limitations

- Computing power, memory and storage at a site are limited compared with a data centre
- Many small units spread over many locations are harder to update, monitor and repair than one data centre
- Physical security is weaker: equipment sits in factories, streets and vehicles rather than in a guarded building
- Devices and gateways from different makers must be made to work together, and standards are still settling
- Only a local view is available, so conclusions that need data from every site still require the cloud

7. Review Questions

CO1 — Describe modern computing approaches and their characteristics

Bloom's Level L1 — Remember, L2 — Understand

1. Define edge computing. Where exactly is “the edge” in a computing system?
2. Define latency and bandwidth, and state the unit in which each is measured.
3. With a neat diagram, explain the basic working model of edge computing.
4. State any four reasons why edge computing is needed.
5. Explain, with one example each, how edge computing helps when the network connection is unreliable and when the data is sensitive.
6. With a neat diagram, explain the architecture of an edge computing system, naming each tier and stating what it does.
7. What is a gateway in an edge computing system, and what work does it perform?

8. Differentiate between cloud computing and edge computing with respect to location of processing, response time, storage and network usage.
9. Explain why edge computing is described as a partner of cloud computing rather than a replacement for it.
10. What is fog computing, and how is it related to edge computing?
11. List any five applications of edge computing, stating in each case why the processing must be done locally.
12. State any four limitations of edge computing.
13. Describe how a traffic camera system would divide its work between the device, edge and cloud tiers.

8. References and Further Reading

Items 1 to 3 are the prescribed books for this module; chapter numbers are indicative. Items 4 to 6 are the papers from which the definitions and the tiered architecture are taken, and item 7 is the standards site for edge computing in mobile networks.

- [1] P. Lea, *IoT and Edge Computing for Architects*, 2nd Edition, Packt Publishing, 2020 — Reference Book 1 for this course; chapters on edge computing and edge architecture.
- [2] R. Buyya, C. Vecchiola, S. T. Selvi, S. Poojara and S. N. Srirama, *Mastering Cloud Computing*, 2nd Edition, Morgan Kaufmann, 2026 — Text Book 2 for this course; used here for the cloud side of the comparison.
- [3] K. Hwang, G. C. Fox and J. J. Dongarra, *Distributed and Cloud Computing: From Parallel Processing to the Internet of Things*, Morgan Kaufmann, 2012 — Text Book 1 for this course; used here for sensors, devices and the growth in the number of connected things.
- [4] W. Shi, J. Cao, Q. Zhang, Y. Li and L. Xu, “Edge Computing: Vision and Challenges”, *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [5] M. Satyanarayanan, “The Emergence of Edge Computing”, *IEEE Computer*, vol. 50, no. 1, pp. 30–39, 2017.
- [6] F. Bonomi, R. Milito, J. Zhu and S. Addepalli, “Fog Computing and Its Role in the Internet of Things”, *Proceedings of the First MCC Workshop on Mobile Cloud Computing*, ACM, 2012, pp. 13–16.
- [7] ETSI, Multi-access Edge Computing (MEC). <https://www.etsi.org/technologies/multi-access-edge-computing>